

Python - 50 Interview Questions with Answers

Python Basics (Questions 1-10)

Q1. What are Python's key features?

Answer:

- **Interpreted:** No compilation needed
- **Dynamically typed:** No type declarations
- **Object-oriented:** Everything is an object
- **High-level:** Abstract from hardware details
- **Extensive libraries:** Rich standard library
- **Cross-platform:** Write once, run anywhere
- **Easy to learn:** Simple, readable syntax
- **Open-source:** Free to use and distribute

Q2. Explain mutable vs immutable data types.

Answer:

Immutable (cannot be changed after creation):

- int, float, bool, str, tuple, frozenset

```
x = 5
x = x + 1 # Creates new object, doesn't modify original
```

Mutable (can be changed):

- list, dict, set, bytearray

```
my_list = [1, 2, 3]
my_list.append(4) # Modifies same object
```

Implications:

```
# Immutable
a = "hello"
b = a
a = "world"
print(b) # "hello" - b unchanged

# Mutable
list1 = [1, 2, 3]
```

```
list2 = list1
list1.append(4)
print(list2) # [1, 2, 3, 4] - list2 also changed
```

Q3. What is the difference between `==` and `is`?

Answer:

`==`: Compares values (equality)

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True (same values)
```

`is`: Compares identity (same object)

```
print(a is b) # False (different objects)

c = a
print(a is c) # True (same object)
```

Memory addresses:

```
print(id(a)) # Memory address
print(id(b)) # Different address
print(id(c)) # Same as a
```

Special case (integer caching):

```
x = 256
y = 256
print(x is y) # True (Python caches small integers -5 to 256)

x = 257
y = 257
print(x is y) # False (not cached)
```

Q4. Explain list comprehensions.

Answer: Concise way to create lists.

Basic syntax:

```
[expression for item in iterable if condition]
```

Examples:

```
# Squares
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Filtering
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]

# Multiple conditions
result = [x for x in range(20) if x % 2 == 0 if x % 3 == 0]
# [0, 6, 12, 18]

# Nested
matrix = [[i+j for j in range(3)] for i in range(3)]
# [[0, 1, 2], [1, 2, 3], [2, 3, 4]]

# With function
words = ["hello", "world"]
upper = [w.upper() for w in words]
# ['HELLO', 'WORLD']
```

Dict/Set comprehensions:

```
# Dictionary
{x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Set
{x % 3 for x in range(10)}
# {0, 1, 2}
```

Q5. What are `*args` and `**kwargs`?

Answer:

`*args`: Variable positional arguments (tuple)

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4)) # 10
print(sum_all(5, 10))     # 15
```

****kwargs**: Variable keyword arguments (dict)

```
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="John", age=30, city="NYC")
# name: John
# age: 30
# city: NYC
```

Combined:

```
def function(a, b, *args, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"kwargs={kwargs}")

function(1, 2, 3, 4, 5, x=10, y=20)
# a=1, b=2
# args=(3, 4, 5)
# kwargs={'x': 10, 'y': 20}
```

Unpacking:

```
values = [1, 2, 3]
print(*values) # 1 2 3

data = {'name': 'John', 'age': 30}
print_info(**data) # Unpacks dict
```

Q6. Explain Python decorators.

Answer: Functions that modify other functions.

Basic decorator:

```
def my_decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper

@my_decorator
```

```
def say_hello():
    print("Hello!")

say_hello()
# Before function
# Hello!
# After function
```

With arguments:

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f"Hello {name}")

greet("John")
# Hello John
# Hello John
# Hello John
```

Common use cases:

```
# Timing
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.2f}s")
        return result
    return wrapper

# Caching
from functools import lru_cache
@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Q7. What is the difference between `append()` and `extend()`?

Answer:

`append()`: Adds single element

```
list1 = [1, 2, 3]
list1.append(4)
print(list1)  # [1, 2, 3, 4]

list1.append([5, 6])
print(list1)  # [1, 2, 3, 4, [5, 6]]
```

`extend()`: Adds multiple elements from iterable

```
list2 = [1, 2, 3]
list2.extend([4, 5, 6])
print(list2)  # [1, 2, 3, 4, 5, 6]

list2.extend("ab")
print(list2)  # [1, 2, 3, 4, 5, 6, 'a', 'b']
```

Equivalent operations:

```
# These are equivalent:
list1.extend([4, 5])
list1 += [4, 5]
list1 = list1 + [4, 5]
```

Q8. Explain lambda functions.

Answer: Anonymous, single-expression functions.

Syntax: `lambda arguments: expression`

Examples:

```
# Basic
square = lambda x: x**2
print(square(5))  # 25

# Multiple arguments
add = lambda x, y: x + y
print(add(3, 4))  # 7

# With map
```

```

numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]

# With filter
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4]

# Sorting
pairs = [(1, 'one'), (2, 'two'), (3, 'three')]
sorted_pairs = sorted(pairs, key=lambda x: x[1])
# [(1, 'one'), (3, 'three'), (2, 'two')]

```

When to use:

- Short, simple operations
- One-time use
- Callbacks

When NOT to use:

- Complex logic (use def)
- Reusability needed
- Multiple statements

Q9. What are generators and `yield`?

Answer: Functions that return iterators, producing values on-demand.

Generator function:

```

def countdown(n):
    while n > 0:
        yield n
        n -= 1

for num in countdown(5):
    print(num)
# 5, 4, 3, 2, 1

```

vs Regular function:

```

# Regular: Creates entire list in memory
def get_numbers(n):
    return [i for i in range(n)]

# Generator: Produces one at a time
def get_numbers_gen(n):

```

```
for i in range(n):  
    yield i
```

Benefits:

- Memory efficient
- Lazy evaluation
- Can represent infinite sequences

Generator expression:

```
# List comprehension (creates list)  
squares_list = [x**2 for x in range(1000000)]  
  
# Generator expression (creates generator)  
squares_gen = (x**2 for x in range(1000000))
```

Example use case:

```
def read_large_file(file_path):  
    with open(file_path) as f:  
        for line in f:  
            yield line.strip()  
  
# Process file line by line without loading all into memory  
for line in read_large_file('huge.txt'):  
    process(line)
```

Q10. Explain exception handling in Python.

Answer:**Basic try-except:**

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero")
```

Multiple exceptions:

```
try:  
    # code  
    pass  
except ValueError:
```

```
print("Value error")
except TypeError:
    print("Type error")
except (KeyError, IndexError):
    print("Key or Index error")
```

Catching all exceptions:

```
try:
    # code
    pass
except Exception as e:
    print(f"Error occurred: {e}")
```

else and finally:

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print(f"Result: {result}") # Runs if no exception
finally:
    print("Cleanup") # Always runs
```

Raising exceptions:

```
def divide(a, b):
    if b == 0:
        raise ValueError("Divisor cannot be zero")
    return a / b

try:
    divide(10, 0)
except ValueError as e:
    print(e)
```

Custom exceptions:

```
class CustomError(Exception):
    pass

raise CustomError("Something went wrong")
```

Data Structures (Questions 11-20)

Q11. Explain dictionaries and their methods.

Answer:

Creation:

```
dict1 = {'a': 1, 'b': 2}
dict2 = dict(a=1, b=2)
dict3 = dict([('a', 1), ('b', 2)])
```

Access:

```
print(dict1['a']) # 1
print(dict1.get('c', 0)) # 0 (default if not found)
```

Methods:

```
# keys(), values(), items()
print(dict1.keys()) # dict_keys(['a', 'b'])
print(dict1.values()) # dict_values([1, 2])
print(dict1.items()) # dict_items([('a', 1), ('b', 2)])

# update()
dict1.update({'c': 3})

# pop()
value = dict1.pop('a') # Removes and returns

# setdefault()
dict1.setdefault('d', 4) # Add if not exists

# fromkeys()
new_dict = dict.fromkeys(['x', 'y'], 0)
# {'x': 0, 'y': 0}
```

Dictionary comprehension:

```
{x: x**2 for x in range(5)}
```

Q12. What are sets and their operations?

Answer:

Creation:

```
set1 = {1, 2, 3, 4}
set2 = set([3, 4, 5, 6])
```

Operations:

```
# Union
print(set1 | set2) # {1, 2, 3, 4, 5, 6}
print(set1.union(set2))

# Intersection
print(set1 & set2) # {3, 4}
print(set1.intersection(set2))

# Difference
print(set1 - set2) # {1, 2}
print(set1.difference(set2))

# Symmetric difference
print(set1 ^ set2) # {1, 2, 5, 6}
```

Methods:

```
set1.add(5)
set1.remove(2) # Raises KeyError if not found
set1.discard(2) # No error if not found
set1.clear()
```

Use cases:

- Remove duplicates
- Membership testing (fast)
- Mathematical set operations

Q13. Explain list slicing.

Answer:

Syntax: `list[start:end:step]`

Examples:

```
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Basic slicing
```

```

print(lst[2:5])    # [2, 3, 4]
print(lst[:3])    # [0, 1, 2]
print(lst[7:])    # [7, 8, 9]
print(lst[:])     # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] (copy)

# Negative indices
print(lst[-3:])   # [7, 8, 9]
print(lst[:-2])  # [0, 1, 2, 3, 4, 5, 6, 7]

# Step
print(lst[::2])   # [0, 2, 4, 6, 8] (every 2nd)
print(lst[1::2]) # [1, 3, 5, 7, 9]
print(lst[::-1]) # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (reverse)

# Assignment
lst[2:5] = [20, 30, 40]

```

Q14. What is the difference between shallow and deep copy?

Answer:

Shallow copy: Copies object, but references nested objects

```

import copy

original = [[1, 2, 3], [4, 5, 6]]
shallow = copy.copy(original)
# or: shallow = original.copy()
# or: shallow = original[:]

shallow[0][0] = 999
print(original[0][0]) # 999 (nested list affected)

```

Deep copy: Recursively copies all objects

```

deep = copy.deepcopy(original)

deep[0][0] = 888
print(original[0][0]) # 999 (original unchanged)

```

Simple types (immutable):

```

# No difference for immutable types
a = [1, 2, 3]
b = a.copy()
a[0] = 10
print(b) # [1, 2, 3] (unchanged)

```

Q15. Explain `enumerate()` and `zip()`.

Answer:

`enumerate()`: Add counter to iterable

```
fruits = ['apple', 'banana', 'cherry']

for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry

# Start from different index
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}: {fruit}")
```

`zip()`: Combine multiple iterables

```
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]
cities = ['NYC', 'LA', 'Chicago']

for name, age, city in zip(names, ages, cities):
    print(f"{name}, {age}, {city}")
# Alice, 25, NYC
# Bob, 30, LA
# Charlie, 35, Chicago

# Unzip
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
numbers, letters = zip(*pairs)
print(numbers) # (1, 2, 3)
print(letters) # ('a', 'b', 'c')
```

Q16-20. [Continue with more DS questions]

Q16: `collections` module (Counter, defaultdict, namedtuple) **Q17:** `itertools` module **Q18:** String methods

Q19: List vs tuple vs set **Q20:** Sorting and sorted()

OOP in Python (Questions 21-30)

Q21. Explain classes and objects.

Answer:

Class definition:

```
class Dog:
    # Class attribute
    species = "Canis familiaris"

    # Constructor
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age

    # Instance method
    def bark(self):
        return f"{self.name} says Woof!"

    # __str__ method
    def __str__(self):
        return f"{self.name} is {self.age} years old"
```

Creating objects:

```
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)

print(dog1.bark()) # Buddy says Woof!
print(dog1) # Buddy is 3 years old
```

Q22. What is inheritance?

Answer:

Single inheritance:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"
```

```
dog = Dog("Buddy")
print(dog.speak()) # Buddy says Woof!
```

Method overriding:

```
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def greet(self):
        return "Hello from Child"
```

super():

```
class Child(Parent):
    def __init__(self, name):
        super().__init__() # Call parent constructor
        self.name = name
```

Q23. Explain encapsulation and access modifiers.

Answer:

Public (default):

```
class MyClass:
    def __init__(self):
        self.public_attr = "public"
```

Protected (single underscore, convention only):

```
class MyClass:
    def __init__(self):
        self._protected_attr = "protected"
```

Private (name mangling with double underscore):

```
class MyClass:
    def __init__(self):
        self.__private_attr = "private"

    def get_private(self):
```

```

        return self.__private_attr

obj = MyClass()
# print(obj.__private_attr) # AttributeError
print(obj.get_private()) # "private"
print(obj._MyClass__private_attr) # Access via name mangling

```

Q24. What are class methods and static methods?

Answer:

Instance method (default):

```

class MyClass:
    def instance_method(self):
        return f"Instance: {self}"

```

Class method (@classmethod):

```

class MyClass:
    count = 0

    @classmethod
    def increment_count(cls):
        cls.count += 1
        return cls.count

# Call without instance
MyClass.increment_count()

```

Static method (@staticmethod):

```

class Math:
    @staticmethod
    def add(x, y):
        return x + y

# Call without instance
print(Math.add(5, 3)) # 8

```

When to use:

- **Instance:** Access instance data
- **Class:** Work with class-level data, alternative constructors
- **Static:** Utility functions, no access to class/instance

Q25. Explain polymorphism.

Answer:

Method overriding:

```
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

shapes = [Rectangle(4, 5), Circle(3)]
for shape in shapes:
    print(shape.area())
```

Duck typing:

```
# If it walks like a duck and quacks like a duck...
def make_sound(animal):
    print(animal.speak())

# Works with any object that has speak()
make_sound(Dog("Buddy"))
make_sound(Cat("Whiskers"))
```

Q26-30. [More OOP topics]

Q26: Magic/Dunder methods (`__str__`, `__repr__`, `__eq__`, etc.) **Q27:** Property decorators **Q28:** Abstract base classes **Q29:** Multiple inheritance and MRO **Q30:** Composition vs inheritance

Advanced Python (Questions 31-40)

Q31. What is the Global Interpreter Lock (GIL)?

Answer: Mutex that allows only one thread to execute Python bytecode at a time.

Implications:

- **CPU-bound tasks:** GIL prevents true parallelism
- **I/O-bound tasks:** GIL released during I/O, less impact

Solutions:

```
# Multiprocessing (separate processes, separate GILs)
from multiprocessing import Pool

def square(x):
    return x * x

with Pool(4) as p:
    results = p.map(square, range(10))

# Threading (for I/O-bound)
from threading import Thread

def io_task():
    # Network request, file I/O, etc.
    pass

threads = [Thread(target=io_task) for _ in range(5)]
for t in threads:
    t.start()
```

Q32. Explain context managers and **with** statement.

Answer:

Purpose: Manage resources (files, locks, connections)

Using with:

```
# Automatically closes file
with open('file.txt', 'r') as f:
    content = f.read()
# File closed here
```

Creating context manager (class):

```
class FileManager:
    def __init__(self, filename):
        self.filename = filename

    def __enter__(self):
        self.file = open(self.filename, 'r')
```

```

        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False # Propagate exceptions

with FileManager('file.txt') as f:
    content = f.read()

```

Using `contextlib`:

```

from contextlib import contextmanager

@contextmanager
def file_manager(filename):
    f = open(filename, 'r')
    try:
        yield f
    finally:
        f.close()

with file_manager('file.txt') as f:
    content = f.read()

```

Q33. What are metaclasses?

Answer: Classes that create classes.

Class creation:

```

# These are equivalent:
class MyClass:
    pass

MyClass = type('MyClass', (), {})

```

Custom metaclass:

```

class Meta(type):
    def __new__(cls, name, bases, attrs):
        # Modify class before creation
        attrs['added_attr'] = 100
        return super().__new__(cls, name, bases, attrs)

class MyClass(metaclass=Meta):
    pass

```

```
print(MyClass.added_attr) # 100
```

Use cases:

- ORM (Django models)
- Validation
- Logging
- Registration

Q34. Explain async/await and asyncio.

Answer:**Basic async function:**

```
import asyncio

async def fetch_data():
    print("Start fetching")
    await asyncio.sleep(2)
    print("Done fetching")
    return {"data": "value"}

# Run
result = asyncio.run(fetch_data())
```

Multiple concurrent tasks:

```
async def main():
    task1 = asyncio.create_task(fetch_data())
    task2 = asyncio.create_task(fetch_data())

    result1 = await task1
    result2 = await task2

    asyncio.run(main())
```

gather:

```
async def main():
    results = await asyncio.gather(
        fetch_data(),
        fetch_data(),
        fetch_data()
    )
```

```
asyncio.run(main())
```

When to use:

- I/O-bound operations
- Network requests
- Database queries
- Not for CPU-bound tasks

Q35-40. [More advanced topics]

Q35: Memory management and garbage collection **Q36:** Type hints and mypy **Q37:** `functools` module (partial, reduce, wraps) **Q38:** Regular expressions **Q39:** Working with files and paths **Q40:** Virtual environments

Python for Data Science/ML (Questions 41-50)

Q41. NumPy basics.

Answer:**Arrays:**

```
import numpy as np

# Creation
arr1 = np.array([1, 2, 3, 4])
arr2 = np.zeros((3, 4))
arr3 = np.ones((2, 3))
arr4 = np.arange(0, 10, 2)
arr5 = np.linspace(0, 1, 5)

# Operations
arr = np.array([1, 2, 3, 4])
print(arr + 2) # [3, 4, 5, 6]
print(arr * 2) # [2, 4, 6, 8]
print(arr ** 2) # [1, 4, 9, 16]

# Slicing
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[0, :]) # [1, 2, 3]
print(matrix[:, 1]) # [2, 5, 8]

# Aggregations
print(arr.sum())
print(arr.mean())
print(arr.std())
```

Q42. Pandas basics.

Answer:**DataFrame creation:**

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'city': ['NYC', 'LA', 'Chicago']
})

# From CSV
df = pd.read_csv('data.csv')
```

Operations:

```
# Selection
print(df['name']) # Column
print(df.loc[0])  # Row by label
print(df.iloc[0]) # Row by index

# Filtering
print(df[df['age'] > 25])

# Groupby
grouped = df.groupby('city')['age'].mean()

# Apply
df['age_squared'] = df['age'].apply(lambda x: x**2)

# Merge
df1.merge(df2, on='key')
```

Q43-50. [More practical topics]

Q43: List performance and optimization **Q44:** Working with JSON **Q45:** Logging in Python **Q46:** Testing with pytest/unittest **Q47:** Package management (pip, requirements.txt) **Q48:** Common built-in functions (map, filter, reduce) **Q49:** F-strings and string formatting **Q50:** Best practices and PEP 8

Quick Interview Tips

1. **Know basics cold:** Data types, control flow, functions
2. **OOP concepts:** Classes, inheritance, polymorphism
3. **Pythonic code:** List comprehensions, generators, decorators
4. **Standard library:** collections, itertools, functools
5. **Data structures:** When to use list vs tuple vs set vs dict

6. **Performance:** Understand time complexity
 7. **Practical experience:** Mention projects, libraries used
-

Good luck with your Python interview! 🚀